

The KV Cache, Deeper

Prefill, decode, and what the cache really saves · video 2:28

This lesson answers one question: what does the KV cache buy, and what does it cost? GPT-2 writes the same 200 tokens with and without it, 2.4 times faster with it; without it, every step recomputes the whole text and gets slower as the text grows. The cache holds keys and values for every layer and token: 73,728 bytes per token. And reading a prompt in one pass (**prefill**) is 64 times faster than one token at a time, which is why **decoding** is the slow part of inference.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 5).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. What the cache saves	▶ 0:08-1:00	2	☐
2. What the cache holds	▶ 1:00-1:17	2	☐
3. Prefill and decode	▶ 1:17-2:19	2	☐

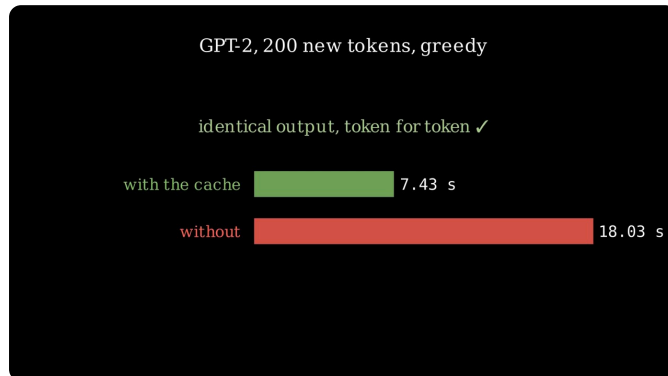
Before you start: LLMs in Practice episode 2 (the KV cache) and Deep Dive episode 9 (grouped-query attention). Code: `code/d31_kv_cache` (GPT-2 small, about 500 MB).

CONCEPT 1 OF 3

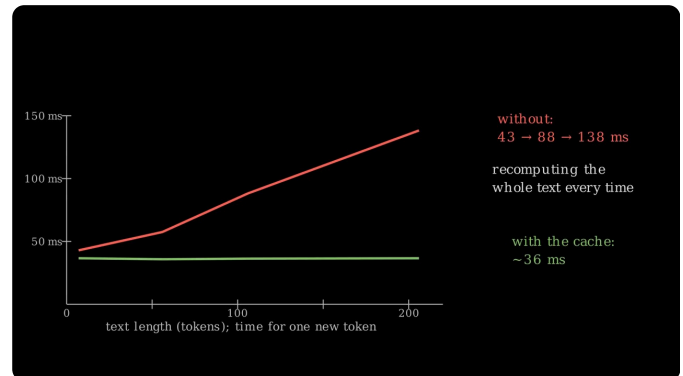
What the cache saves

▶ Video 0:08–1:00

To write each new token, a model needs the keys and values of every token before it. The **KV cache** keeps them instead of recomputing them. GPT-2 writes 200 tokens greedily, twice: the output is identical token for token, but it takes **7.43 s** with the cache and **18.03 s** without, 2.4 times slower.



▶ 0:39 200 tokens, identical output: 7.43 s with the cache, 18.03 s without.



▶ 0:58 Time per new token: ~36 ms with the cache; without, 43 → 88 → 138 ms as the text grows.

Step by step: with the cache every new token costs about **36 ms**, however long the text. Without it the cost keeps growing: 43 ms at 7 tokens, 57.5 at 56, 88 at 106, 138 at 206, because the whole text is recomputed every time.

KEY IDEA

The cache turns each new token's cost from “the whole text” into “one token plus a lookup”; the output is exactly the same.

Check 1: What the cache saves

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

With the cache, about how many tokens per second does GPT-2 generate here?

.....

1.2 SHORT ANSWER

Why does the time per token without the cache grow roughly in proportion to the text length?

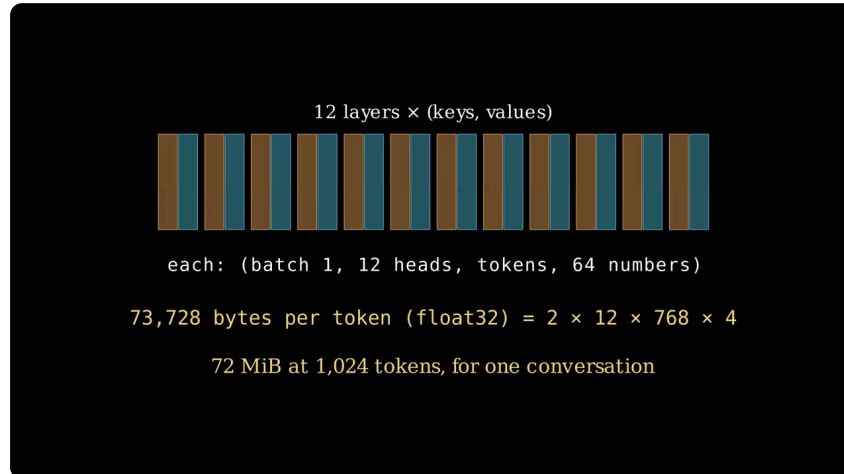
.....

CONCEPT 2 OF 3

What the cache holds

▶ Video 1:00-1:17

For each of GPT-2's 12 layers the cache holds keys and values of shape (batch 1, 12 heads, tokens, 64 numbers). That is $2 \times 12 \times 768 \times 4 = \mathbf{73,728 \text{ bytes per token}}$ in float32: **72 MiB** at 1,024 tokens, for one conversation.



▶ 1:16 12 layers × keys and values of shape (1, 12, tokens, 64):
73,728 bytes per token; 72 MiB at 1,024 tokens.

KEY IDEA

Cache size = $2 \times \text{layers} \times (\text{key/value heads} \times \text{head size}) \times \text{bytes} \times \text{tokens}$: it grows with every token and every user.

Check 2: What the cache holds

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

GPT-2's cache in float16: how many bytes per token, and how many MiB at 1,024 tokens?

2.2 MULTIPLE CHOICE

Qwen2.5-0.5B's bfloat16 cache is 12,288 bytes per token (episode 9); GPT-2's in float16 is 36,864. Why is the larger model's cache smaller?

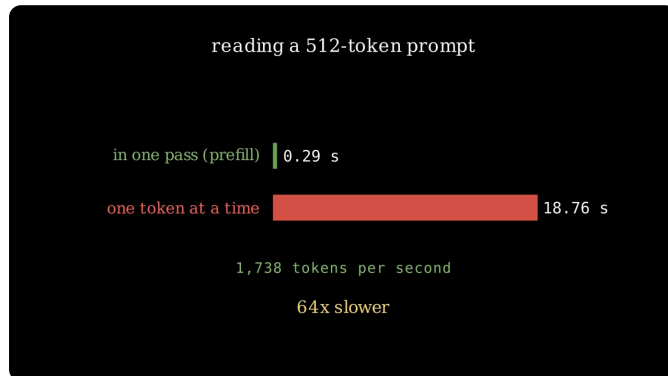
- A. It has fewer layers
- B. Grouped-query attention: 2 key/value heads instead of one per query head
- C. It uses shorter tokens
- D. It does not cache values

CONCEPT 3 OF 3

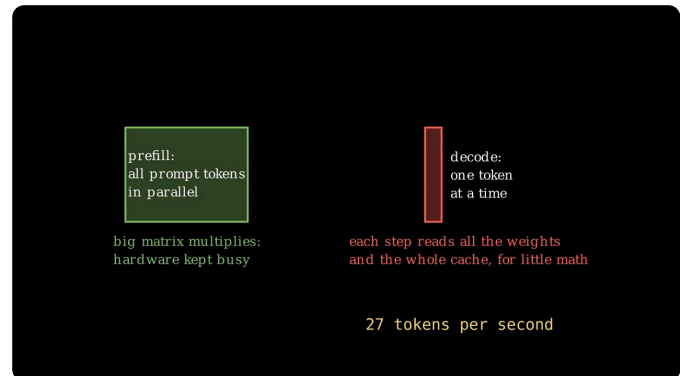
Prefill and decode

▶ Video 1:17-2:19

Reading a 512-token prompt in one pass, called **prefill**, takes **0.29 s**: 1,738 tokens per second. Feeding the same prompt one token at a time takes **18.76 s**, 64 times slower. Prefill processes all the prompt's tokens in parallel, with big matrix multiplications that keep the hardware busy. **Decoding** can only produce one token at a time, and each step reads all the weights and the whole cache for very little math: here, 27 tokens per second.



▶ 1:34 A 512-token prompt: 0.29 s in one pass (1,738 tokens/s), 18.76 s one token at a time.



▶ 1:52 Prefill keeps the hardware busy; each decode step reads all the weights and the cache for little math.

That is why a chatbot pauses before its answer (prefill) and then streams it word by word (decode), and why the cache's size limits context length and the number of users per machine. Shared keys and values (episode 9) and sliding windows (episode 11) shrink it. In code, the cache is `past_key_values`: pass it back in and feed only the newest token.

KEY IDEA

Prefill is compute-bound and fast per token; decode is memory-bound and slow per token.

Check 3: Prefill and decode

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

At the measured rates, how long would prefill of a 2,000-token prompt take, and how long to decode a 200-token answer?

3.2 TRUE OR FALSE

“Generating tokens is slow mainly because each step does a lot of arithmetic.”

True ☐ False ☐

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. What the cache saves · rewatch ▶ 0:08-1:00

1.1 About 27. 200 tokens / 7.43 s $\approx$ 26.9.

1.2 Each step runs the model over the whole text so far, so the work grows with the number of tokens (and attention even with its square). 43 $\rightarrow$ 57.5 $\rightarrow$ 88 $\rightarrow$ 138 ms at 7, 56, 106, 206 tokens.

2. What the cache holds · rewatch ▶ 1:00-1:17

2.1 36,864 bytes; 36 MiB. Half of 73,728 bytes per token.

2.2 B. 2 K/V heads $\times$ 64 $\times$ 24 layers vs 12 heads $\times$ 64 $\times$ 12 layers.

3. Prefill and decode · rewatch ▶ 1:17-2:19

3.1 About 1.2 s and 7.4 s. 2,000 / 1,738 $\approx$ 1.15 s; 200 / 27 $\approx$ 7.4 s (on this CPU; real servers are far faster but show the same imbalance).

3.2 False. Each step does little math for the data it must read; it is limited by moving the weights and cache, not by arithmetic.